

PRD

Consulting Platform — Production Grade Backend PRD

Project Type

AI-Powered Consultancy Management Platform

1. Product Overview

The platform is a production-grade consultancy management system designed for consultants, clients, and administrators.

The system provides:

- Authentication & authorization
- Consultant management
- Consultation booking system
- Transaction management
- AI-powered chatbot assistance
- Analytics & reporting
- Review & rating management
- Admin management system
- Audit logging
- Notification system

The backend architecture follows enterprise-level engineering standards using layered architecture and clean separation of concerns.

2. Core Modules

2.1 Authentication & Authorization Module

Features

- JWT authentication
- Refresh token management
- Role-based access control
- Password hashing
- Email verification
- Password reset workflow
- Session management
- Secure token rotation

Roles

- Client
- Consultant
- Admin
- Super Admin

APIs

Auth APIs

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh-token
- POST /api/v1/auth/logout
- POST /api/v1/auth/forgot-password

- POST /api/v1/auth/reset-password
 - GET /api/v1/auth/me
-

2.2 User Management Module

Features

- User profile management
- Role assignment
- Account activation/deactivation
- User listing with pagination
- Search & filtering
- Soft delete support

APIs

- GET /api/v1/users
 - GET /api/v1/users/{id}
 - PUT /api/v1/users/{id}
 - DELETE /api/v1/users/{id}
-

2.3 Consultant Management Module

Features

- Consultant onboarding
- Expertise management
- Experience management
- Availability scheduling
- Pricing configuration
- Rating aggregation

- Consultant search & filtering

APIs

- POST /api/v1/consultants
 - GET /api/v1/consultants
 - GET /api/v1/consultants/{id}
 - PUT /api/v1/consultants/{id}
 - DELETE /api/v1/consultants/{id}
-

2.4 Consultation Booking Module

Features

- Consultation scheduling
- Consultation status tracking
- Rescheduling workflow
- Cancellation workflow
- Consultation history
- Slot validation
- Conflict prevention

Consultation Status

- Pending
- Confirmed
- In Progress
- Completed
- Cancelled

APIs

- POST /api/v1/consultations
 - GET /api/v1/consultations
 - GET /api/v1/consultations/{id}
 - PUT /api/v1/consultations/{id}
 - DELETE /api/v1/consultations/{id}
-

2.5 Transaction & Payment Module

Features

- Payment tracking
- Transaction history
- Revenue calculation
- Invoice generation
- Payment status tracking

APIs

- POST /api/v1/transactions
 - GET /api/v1/transactions
 - GET /api/v1/transactions/{id}
 - GET /api/v1/transactions/summary
-

2.6 Review & Rating Module

Features

- Consultant reviews
- Star ratings
- Feedback moderation
- Average rating calculation

- Review listing

APIs

- POST /api/v1/reviews
 - GET /api/v1/reviews
 - GET /api/v1/reviews/{consultant_id}
 - DELETE /api/v1/reviews/{id}
-

2.7 AI Chatbot Module

Features

- Natural language query handling
- Intent classification
- SQL query generation
- Consultation assistance
- Analytics assistance
- Chat history management
- Role-aware responses

APIs

- POST /api/v1/chat/message
 - GET /api/v1/chat/history
 - DELETE /api/v1/chat/history
-

2.8 Analytics Module

Features

- Total users analytics

- Revenue analytics
- Booking analytics
- Consultant performance analytics
- Consultation trends
- Dashboard metrics

APIs

- GET /api/v1/analytics/dashboard
 - GET /api/v1/analytics/revenue
 - GET /api/v1/analytics/consultations
-

2.9 Notification Module

Features

- Email notifications
 - Booking notifications
 - Payment notifications
 - Reminder notifications
 - System alerts
-

2.10 Audit Logging Module

Features

- User activity tracking
- API request logging
- Authentication logs
- Error logging
- Security event logging

3. Production Architecture

The application follows a layered production-grade backend architecture.

Client

↓

API Router Layer

↓

Controller / Handler Layer

↓

Service Layer

↓

Repository Layer

↓

Database Layer

4. Project Structure

app/

|

├── api/

|

├── v1/

|

│ └── auth/

|

│ └── users/

|

│ └── consultants/

|

│ └── consultations/

|

│ └── transactions/

|

│ └── reviews/

```
| | | — analytics/
| | | — chatbot/
| |
| | — dependencies/
| | — middlewares/
| | — routers.py
|
| — core/
| | — config.py
| | — security.py
| | — constants.py
| | — permissions.py
| | — exceptions.py
| | — logger.py
| | — settings.py
|
| — db/
| | — session.py
| | — base.py
| | — migrations/
| | — seeders/
|
| — models/
| | — user.py
| | — consultant.py
| | — consultation.py
| | — transaction.py
```

```
| |— review.py
| |— notification.py
| |— audit_log.py
| |— chat_message.py
|
|— schemas/
| |— auth/
| |— users/
| |— consultants/
| |— consultations/
| |— transactions/
| |— reviews/
| |— analytics/
| |— chatbot/
|
|— repositories/
| |— base_repository.py
| |— user_repository.py
| |— consultant_repository.py
| |— consultation_repository.py
| |— transaction_repository.py
| |— review_repository.py
| |— analytics_repository.py
```

5. Repository Layer Responsibilities

- Database abstraction

- CRUD operations
 - Query optimization
 - Pagination queries
 - Filtering queries
 - Transaction handling
 - Reusable database operations
-

6. Service Layer Responsibilities

- Business logic implementation
 - Validation handling
 - Workflow orchestration
 - Transaction management
 - External service integration
 - AI processing logic
 - Permission handling
-

7. API Layer Responsibilities

- Request validation
 - Dependency injection
 - Authentication checks
 - Response formatting
 - API versioning
 - Route management
-

8. Database Design

Tables

users

- id
- name
- email
- password_hash
- role
- is_active
- created_at
- updated_at

consultants

- id
- user_id
- expertise
- experience
- pricing
- availability
- average_rating

consultations

- id
- client_id
- consultant_id
- scheduled_at
- status

- notes

transactions

- id
- consultation_id
- amount
- payment_status
- transaction_reference

reviews

- id
- consultant_id
- client_id
- rating
- comment

chat_messages

- id
- user_id
- message
- response
- intent

audit_logs

- id
- user_id
- action
- ip_address

- created_at
-

9. Tech Stack

Backend

- FastAPI
- Python
- SQLAlchemy Async
- Alembic
- Pydantic

Database

- PostgreSQL

Authentication

- JWT
- bcrypt

Caching

- Redis

AI Integration

- OpenAI API
- Intent classification service

Background Jobs

- Celery
- Redis Queue

DevOps

- Docker
 - Docker Compose
-

10. Security Requirements

- JWT authentication
 - Password hashing
 - Role-based authorization
 - API rate limiting
 - Secure headers
 - Input validation
 - SQL injection prevention
 - Environment variable protection
 - CORS configuration
-

11. Non-Functional Requirements

Performance

- Async request handling
- Optimized database queries
- Connection pooling
- Pagination support

Scalability

- Modular architecture
- Stateless APIs

- Repository abstraction
- Service-based separation

Maintainability

- Clean code structure
- Layer separation
- Reusable components
- Standardized responses

Reliability

- Exception handling
 - Structured logging
 - Retry mechanisms
 - Health check endpoints
-

12. Additional Production Features

- Centralized exception handling
- API versioning
- Structured logging
- Dependency injection
- Environment-based configuration
- Health monitoring
- Soft deletes
- Database migrations
- Seed scripts
- Unit testing support

- Integration testing support
 - Reusable response schemas
 - Global validation handling
-

13. Deployment Architecture

Client

↓

Nginx

↓

FastAPI Application

↓

Redis

↓

PostgreSQL

14. Development Phases

Phase 1

- Project setup
- Database setup
- Authentication system
- User management

Phase 2

- Consultant management
- Consultation booking

- Transactions
- Reviews

Phase 3

- AI chatbot
- Analytics
- Notifications
- Audit logs

Phase 4

- Dockerization
 - Redis integration
 - Background jobs
 - Testing
 - Production deployment
-

15. Final Objective

Build a production-ready consultancy backend system using enterprise backend architecture patterns while maintaining clean scalability, modularity, maintainability, and proper engineering standards.

- Proper enterprise backend architecture
- Layered MVC + Service + Repository flow
- Full production folder structure
- API versioning
- Modular services
- Security requirements
- Audit logging

- Notification module
- Redis + Celery architecture
- Deployment architecture
- Testing structure
- Background tasks
- Scalable database design
- Clean REST API naming
- No beginner-style examples